

The SQL Injection Attack

Dean Safran

May 23, 2026

1 Abstract

This paper, and repository demo that goes along with it, demonstrates sequel (SQL) injection as a broad security vulnerability class as well as an isolated exploit on a music streaming web app. The class of SQL injection attacks are a security vulnerability where an individual manipulates a database via injecting malicious SQL code into user controlled fields. This vulnerability arises when applications create SQL statements at runtime via raw string concatenation rather than treating user input as data, separate from executable commands, that is later binded into the generic template SQL queries. Due to the fact that databases store a myriad of sensitive information like usernames, passwords, financial data, and private records, SQL injection attacks leave many openings for attackers to steal, modify or destroy confidential data. Although SQL is one of the oldest security exploits, it is still very relevant to modern technology as databases are widely used in modern software development.

2 What is the vulnerability or countermeasure?

The vulnerability within databases is within the backend implementation. When a programmer allows user input to become part of the database query without separating and cleaning user data beforehand (similar to buffer overflows when not checking the length of inputs). In these types of systems, attackers can put their malicious SQL code into login forms, search bars, or URL parameters which allows unintended commands to be executed on the database. A vulnerable query frequently includes string concatenation as follows:

```
const sql = `
  SELECT * FROM Song
  WHERE songName = '${songName}'
`;
```

If a user enters:

Video Games' OR '1'='1

The resulting SQL turns into:

```
SELECT * FROM Song
WHERE songName = 'Video Games' OR '1'='1'
```

Originally, the programmer intended for only a few songs to be outputted. Instead, our malicious code returns every row because $1=1$ is always true. Here it's evident the core vulnerability does not lie within the database, but the unsafe creation of queries in the backend that handles user input as executable code instead of plain text data.

In order to combat this, developers should use parameterized queries, also known as 'bind variables' or 'prepared statements'. Instead of user inputs being concatenated to string variables containing SQL statements, the backend includes placeholders in javascript such as `:songName`, then proceeds to send user input separately. For example,

```
WHERE songName = :songName
```

Signals to the database 'songName' is data, not executable SQL code. Inputs such as

"Video Games' OR '1'='1"

Is treated as one literal song name, which would be strange, but in today's society ... *throws hands up* *guffaws*. This beats the alternative where this input is treated as logic which changes the WHERE clause which is the clause that filters what data is presented to the user. Other defenses include validating inputs, escaping special characters, limiting database permission, and avoiding detailed error messages according to these articles.¹

In our implementation of our demo repository, bind variables were the most widely implemented fix.

3 What resources are affected?

A SQL injection attack targets the underlying relational database and any resources tied to it. Examples of these resources include, but are not limited to, financial information, email addresses, private messages, authentication tokens, etc. In bigger databases in large corporations, attackers may also inject code that displays internal business records, intellectual property, or administrative account information. In addition to data based resources, the backend component of a web app server may also become compromised as attackers can bypass login systems and execute administrative operations. Occasionally these types of attacks can also go as deep as touching the operating system itself via stored procedures, or if the database itself was privileged.

4 In what way are resources affected? (confidentiality, integrity/authorization, availability, non-repudiation)

A SQL injection attack affects all parts of the CIAA security model. Confidentiality is compromised when attackers use queries to read stored usernames and passwords in the

¹https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

<https://portswigger.net/web-security/sql-injection>

<https://cwe.mitre.org/data/definitions/209.html>

<https://learn.microsoft.com/en-us/sql/relational-databases/security/sql-server-security-best-practices?view=sql-server-ver17>

database. Integrity and authorization are affected when hackers use INSERT, DELETE, or other queries that alter data, or use newfound data to gain unauthorized privileges. Availability may be at risk if an exploit executes many queries such that the database becomes overloaded, or if queries such as DELETE on tables with ON DELETE CASCADE to destroy many tables. Lastly, non repudiation is affected when attackers impersonate users by using their injected SQL to extract user-passwords, then perform actions under another account.

5 Who would gain from undertaking such an attack/defense? What kind of background or resources does that person need?

Many types of attackers can benefit from performing a SQL injection attack. Cybercriminals partake in injection attacks to extract passwords, credit card information, or personal data that can later be sold on the black market. Political groups and hacktivists may partake in SQL injection attacks to deface websites by leaking sensitive data. Nation state actors may use it for espionage against universities, corporations, or even governments. Inexperienced coders, sometimes denoted as ‘script kiddies’, can execute a SQL injection using tools a google search away, learning from tutorials because the vulnerability is so well documented.

One of the reasons SQL has remained particularly vulnerable and dangerous is the technical barrier to entry to relatively low. An attacker only needs a browser, simple understanding of SQL syntax, and a clue as to how web forms interact with databases. In our demo repository, only had three javascript layers separated the front end that users interacted with from the database itself. Tools such as sqlmap² can automate much of the attack process. It can detect vulnerabilities, and act accordingly in order to extract data. More advanced attacks would require deeper knowledge of a web app’s internals and web application structures when targeting protected systems or attempting privilege escalation.

6 Is the attack/defense difficult?

In most cases, these attacks are relatively simple to perform. In the example above, a simple extra piece of information,

```
' OR '1'='1
```

Was needed to make a query corrupted. Because many applications relied on string concatenation to construct SQL queries, attackers only needed a smidge more than experimentation and patience to crack a database. However, modern applications have mutated SQL injection attacks into a significantly more difficult class of computer security attacks. Defenses such as firewalls, rate limiting, and restricted database permissions require new strategies such as bypassing filters, inferring database behavior through timed attacks, or exploiting “blind” SQL vulnerabilities (when error messages and outputs are hidden), respectively.

²<https://sqlmap.org/>

Defending against these attacks is straightforward in theory, but practically challenging. When developers create gigantic databases and backend infrastructure to access them, constant use of secure coding practices makes or breaks security. Using proper syntax across every database query in a large application is challenging, and a single insecure query can leave the entire database vulnerable. In our application, only a couple files with a couple javascript functions that connect to the database via a couple API endpoints were susceptible to this attack. If these were scattered randomly amongst the other functions in their files, they would be a bit difficult to find. Our project was only a hundredth of the size of a real database, imagine protecting these functions when they exist amongst hundreds of other files.

7 Are the consequences of the attack/defense mild or severe?

The consequences of a SQL injection attack are often severe due to the fact that databases store an organization's most valuable information. A successful attack can leak passwords, financial records, medical data, private communications, or virtually anything that uses data. In more serious infiltrations, exploits can delete tables, bypass authentication systems, or gain administrative control over databases.

8 What is the history of this attack/defense? Who invented it (if we know)? Is it hypothetical or “in the wild”? Has it ever been deployed maliciously?

Exploiting databases via an injection attack is one of the oldest and most famous web application attacks. This class of attacks is most certainly ‘in the wild’ as it became most widely recognized in the late 1990s and early 2000s as web applications frequently relied on database-driven architectures but generated sequel queries unsafely by implementing string concatenation.

There is no single ‘inventor’ of this class of exploit in the normal sense. It is better put to describe it as first publicly documented and recognized by Jeff Forristal, also known as rain.forest.puppy. In a Phrack article dated all the way back to 1998, Black Hat gives him the accolade of ‘first publicized recognition of SQL injection’.³

A major turning point for this class of security vulnerability was in 2002, when researcher Chris Anley published “Advanced SQL Injection in SQL Server Applications”. The paper portrayed SQL injection as more than a login-bypass trick. Anley showed attackers could execute administrative database procedures such as `xp_cmdshell`, allowing the SQL Server to run operating system commands directly. The paper warns that database vulnerabilities could lead to ‘full control over the machine hosting the database server’, which completely shifted the outlook on how researchers viewed this type of attack.⁴

³<https://blackhat.com/us-13/speakers/Jeff-Forristal.html>

⁴https://www.cgisecurity.com/lib/advanced_sql_injection.pdf

Only 9 years later, in 2011, hacker group LulzSec proclaimed that they found the extremely rudimentary flaw of database string concatenation, found in the demo repository, in SonyPictures.com. According to Wired, LulzSec reported they claimed over 1 million accounts were leakable, despite the fact they only released 50,000 records due to “a lack of resources.” The stolen data reportedly included emails and passwords from Sony’s promotional contest databases. LulzSec publicly mocked Sony following the attack, in essence, they stated ‘they had one of the most insecure websites we’ve ever seen...’.⁵

This was not a laughing matter, however, as one group member was sentenced to pay over \$605,663 in restitution.⁶

9 How Does It Work? In Detail...

The intuition behind SQL injection is essentially this: the program accidentally allows the user to edit the string query sent to the database. The developer intends the user is only filling in a blank, such as the following:

Show me menu items from dining hall: ____

But the vulnerable program, in reality, instead copies user text into a command:

```
SELECT menu items
WHERE hall_name = '____'
AND visible_to_students = 'Y'
```

That difference is the whole vulnerability. The user isn’t just providing a fill in the blank answer, they are filling in the missing piece of logic from a database command itself.

Let us consider a pedagogical example of a campus dining app (like our lovely WSO) that should only list menu items marked as public:

```
function getVisibleMenu(hallName) {
  const sql =
    "SELECT item_name, station, calories " +
    "FROM CampusMenu " +
    "WHERE hall_name = '" + hallName + "' " +
    "AND visible_to_students = 'Y'";

  return db.execute(sql);
}
```

A normal input would most likely be:

Whitmans

⁵<https://itwire.com/business-it-news/security/sony-falls-victim-to-another-simple-sql-injection-attack/>
<https://www.computerworld.com/article/1530842/sony-pictures-falls-victim-to-major-data-breach-2.html>

<https://www.wired.com/2011/06/sony-lulzsec/>

⁶<https://www.justice.gov/usao-cdca/pr/member-lulzsec-hacking-group-sentenced-over-year-federal-prison>

The generated SQL turns into the following:

```
SELECT item_name, station, calories
FROM CampusMenu
WHERE hall_name = 'Whitmans'
AND visible_to_students = 'Y'
```

This step is what the developer wanted to happen. The database solely returns the public menu items from Whitmans. But now let us analyze the hypothetical where the user enters:

Whitmans' –

The generated SQL is:

```
SELECT item_name, station, calories
FROM CampusMenu
WHERE hall_name = 'Whitmans' --'
AND visible_to_students = 'Y'
```

In SQL, `--` begins a comment, which means the SQL engine ignores the rest of the line. The attacker has effectively removed the condition:

```
AND visible_to_students = 'Y'
```

The following result the malicious attacker has gleaned from the database may be all Whitmans menu items, including hidden rows. The attacker didn't need any login information, no rainbow tables, nothing. They only needed to tweak a string sent to the database.

The database only receives the final string, it cannot differentiate between characters written by honest clients versus malicious hackers. Once the string reaches the database, everything is parsed as SQL.

Formally, the vulnerable program constructs a query like this:

$$Q(x) = A || x || B$$

Where x is user input, and A and B are trusted SQL pieces written by the developer. x is presumed to be a literal value of data. The issue is that SQL is a programming language with grammar. This means quotes, comments, keywords, parentheses, and operators all affect how the end resulting string is interpreted. The vulnerability occurs when:

$$\exists x \text{ such that } \text{parse}(A||x||B) \neq \text{intended_parse}(A||\text{literal}(x)||B)$$

In human terms: the database's interpretation of the query is different than was meant to occur. SQL injection is fundamentally a code-versus-data confusion. The application should treat user input as data (like we do in the demo by treating front end JSON as data placed into bind variables), but instead lets that input become code.

SQL injection also extends its reach to computational resources. A safe lookup into a relational database may be relatively inexpensive. If `hall_name` is indexed, for example, the database might find matching rows in about $O(\log n)$ time, where n is the number of menu rows. However, if injection removes a filtering condition more rows may be returned,

potentially closer to $O(n)$. This leads to more CPU time being spent scanning or returning rows, more memory storing intermediate results (intermediate steps from JOIN in SQL involve the cartesian product of databases, without filtering rows out, that's a lot of extra rows!), and more network bandwidth sending the response back to the application. This transforms a security flaw into a performance flaw too.

We can also frame this probabilistically. Suppose an application has m different places where user input touches SQL queries. If each query has probability p of being written vulnerably, then the expected amount of vulnerable queries is:

$$E = m \cdot p$$

Even if p is small, a large application may have thousands of queries, and therefore there would be meaningful risk. The probability that at least one SQL injection vulnerability exists is:

$$1 - (1 - p)^m$$

As m grows, this probability grows. This is why SQL injection remains dangerous in big systems. One careless query can expose an otherwise safe application, such as the demo repo.

The best defense against this is not to guess the characters we believe to be dangerous. The better strategy is to prevent user input from being interpreted as code at all. This is done with parameterized queries. The safe version of the hall example would look like this:

```
function getVisibleMenuSafe(hallName) {
  const sql =
    "SELECT item_name, station, calories " +
    "FROM CampusMenu " +
    "WHERE hall_name = :hallName " +
    "AND visible_to_students = 'Y'";

  return db.execute(sql, { hallName: hallName });
}
```

Here, `:hallName` is a placeholder. The SQL structure is kept separate from the user's value. If the user types:

Whitmans' –

The database treats the entire input as a literal dining hall name. It doesn't treat the quote or dashes as the beginning of a comment as in the string concatenation version.

10 Countermeasures/counterattacks. You should describe the best known countermeasures/counterattacks, if there are any.

The main countermeasure, as described in the previous section, is bind variables, or parameterized queries, which clearly delineate between user input and SQL code. Other defenses

can assist in preventing injection attacks as well. ‘Input validation’ rejects strange or unexpected input, like requiring zip code to be only numbers. ‘Escaping’ places a protective marker before special character, such as quotes, so they are read as text rather than SQL syntax. ‘Least privilege’ stands for when a web app’s database account should only have permissions it needs. An example of our to structure a database with least privilege could be implementing search features with no permissions to delete tables. Safe error messages hide detailed database errors from users (try entering in no values in our database, you get an error message hand crafted by yours truly, no SQL ERROR output).

11 Conclusion. What are the broader implications of this attack for technology and society?

SQL injection demonstrates how a small structural change in database infrastructure design can become a major vulnerable exploit. Technically, the bug is simple: a program fails to separate user input from database code. Consequences, however, span far more than one broken query. One vulnerable search box, one incorrectly used login form can lead to the exposing of private financial, medical, or institutional secrets. The broader implication of this class of exploit in today’s society is how a plethora of applications are reliant on invisible database systems day to day users can never see or audit themselves. Users have inherent trust in websites, schools, banks, hospitals, and companies to safeguard their data. SQL injection shatters this trust by exposing careless software design, where developers don’t build with malicious user input in mind. As user data becomes more of a commodity – such as investors and countries bidding for tiktok’s incredible ability to store and use user data⁷ – protecting the boundary between code and data is absolutely integral to the safety of peoples’ lives stored inside them.

⁷<https://www.reuters.com/sustainability/boards-policy-regulation/tiktok-lives-us-china-deal-app-keep-operating-us-2025-09-16/>